Circuit Breaker

Circuit Breaker · Protect calls to flaky rates API · Build with Waqar

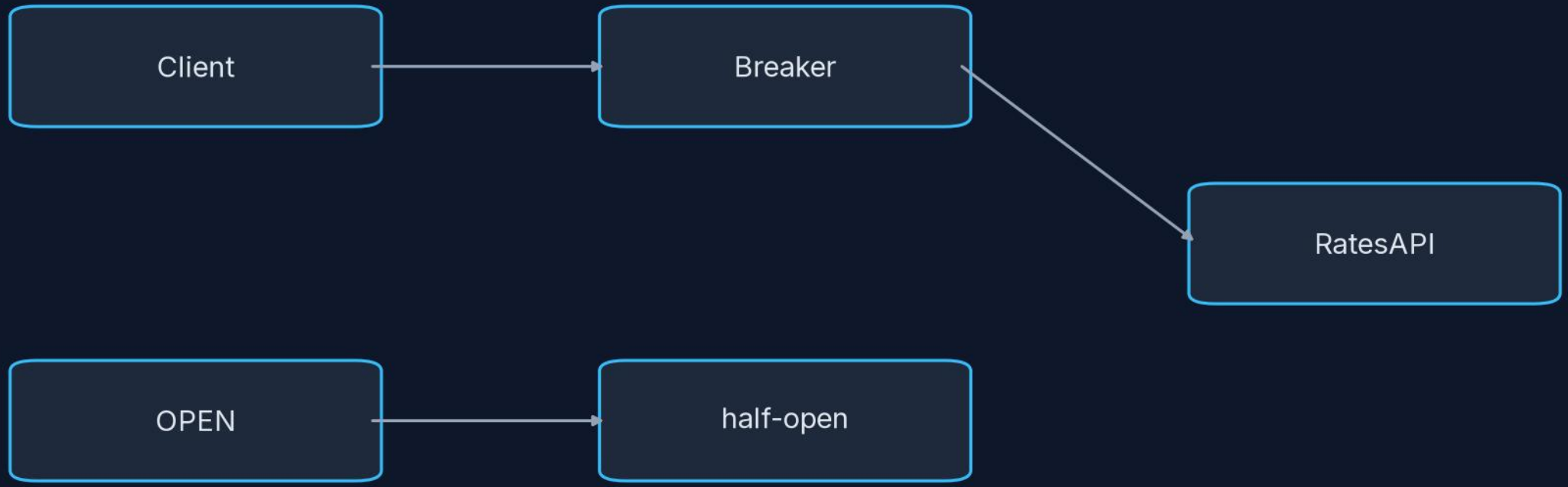
Today's Hook

- Problem: Protect calls to flaky rates API
- Pattern: Circuit Breaker — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Protect calls to flaky rates API
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Circuit Breaker: Protect calls to flaky rates API

Real-World Scenario

- Running example: Protect calls to flaky rates API
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.enterprise.circuitbreaker;

import com.buildwithwaqar.patterns.enterprise.circuitbreaker.CircuitBreaker;
import java.time.*;
import java.util.concurrent.atomic.AtomicReference;
/** Problem: stop hammering a flaky rates HTTP API; recover after timeout. */
public final class UseCaseDemo {
  public static void main(String[] a) throws Exception {
    System.out.println("=== Circuit Breaker problem: flaky rates API ===");
    var now = new AtomicReference<>(Instant.parse("2026-01-01T00:00:00Z"));
    var cb = new CircuitBreaker(2, Duration.ofSeconds(30), now::get);
    for (int i=0;i<2;i++) try { cb.call(() -> { throw new RuntimeException("503"); }); } catch (Exception e) { System.out.println("fail state="+cb.state()); }
    try { cb.call(() -> "ok"); } catch (Exception e) { System.out.println("shortCircuit="+e.getMessage()); }
    now.set(now.get().plusSeconds(31));
    System.out.println("recovered="+cb.call(() -> "rate=83.2")+" state="+cb.state());
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.enterprise.circuitbreaker;

import java.time.Duration;
import java.time.Instant;
import java.util.concurrent.Callable;
import java.util.function.Supplier;

public final class CircuitBreaker {
    public enum State { CLOSED, OPEN, HALF_OPEN }

    private final int failureThreshold;
    private final Duration openTimeout;
    private final Supplier<Instant> clock;

    private State state = State.CLOSED;
    private int failures;
    private Instant openedAt = Instant.EPOCH;

    public CircuitBreaker(int failureThreshold, Duration openTimeout, Supplier<Instant> clock) {
        this.failureThreshold = failureThreshold;
        this.openTimeout = openTimeout;
        this.clock = clock;
    }
```

Solution (2)

```
    public State state() { return state; }

    public <T> T call(Callable<T> action) throws Exception {
        if (state == State.OPEN) {
            if (clock.get().isAfter(openedAt.plus(openTimeout))) {
                state = State.HALF_OPEN;
            } else {
                throw new IllegalStateException("circuit open");
            }
        }
        try {
            var result = action.call();
            failures = 0;
            state = State.CLOSED;
            return result;
        } catch (Exception e) {
            failures++;
            if (failures >= failureThreshold || state == State.HALF_OPEN) {
                state = State.OPEN;
                openedAt = clock.get();
            }
```

Solution (3)

```
                throw e;
            }
        }
    }
```

Runnable main

```
package com.buildwithwaqar.patterns.enterprise.circuitbreaker;

import com.buildwithwaqar.patterns.enterprise.circuitbreaker.CircuitBreaker;
import java.time.*;
import java.util.concurrent.atomic.AtomicReference;
/** Problem: stop hammering a flaky rates HTTP API; recover after timeout. */
public final class UseCaseDemo {
  public static void main(String[] a) throws Exception {
    System.out.println("=== Circuit Breaker problem: flaky rates API ===");
    var now = new AtomicReference<>(Instant.parse("2026-01-01T00:00:00Z"));
    var cb = new CircuitBreaker(2, Duration.ofSeconds(30), now::get);
    for (int i=0;i<2;i++) try { cb.call(() -> { throw new RuntimeException("503"); }); } catch (Exception e) { System.out.println("fail state="+cb.state()); }
    try { cb.call(() -> "ok"); } catch (Exception e) { System.out.println("shortCircuit="+e.getMessage()); }
    now.set(now.get().plusSeconds(31));
    System.out.println("recovered="+cb.call(() -> "rate=83.2")+" state="+cb.state());
  }
}
```

Walkthrough Output

```
=== Circuit Breaker problem: flaky rates API ===
fail state=CLOSED
fail state=OPEN
shortCircuit=circuit open
recovered=rate=83.2 state=CLOSED
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Circuit Breaker.

Related Patterns

- Contrast Circuit Breaker with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Circuit Breaker → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.